192524146_Jaya shree_Co1_AT2

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyse DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships.* (BL1, BL2)

Activity Tool : Concept Mapping (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand
3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand

6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand
7	Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.	BL2 – Understand
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand
10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember

---

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts

Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

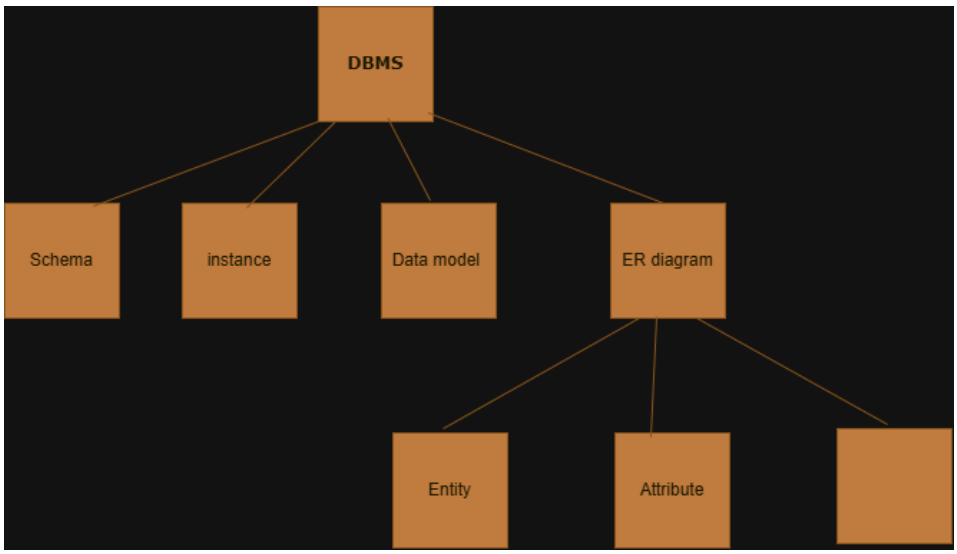
1.Construct a Concept Map Linking the Following Terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.

Introduction

A **Database Management System (DBMS)** is software that allows users to create, store, organize, retrieve, and manage data efficiently. It provides an organized way of handling large amounts of data while ensuring security, consistency, and integrity.

The **Data Model** defines how data is organized within a database. During database design, an **Entity-Relationship (ER) Diagram** is created using entities, attributes, and relationships. The final database structure is called the **Schema**, while the actual data stored at a particular time is called the **Instance**.

Concept Map



Explanation of the Concept Map

1. DBMS (Database Management System)

Definition

A **DBMS** is software that manages databases and provides facilities to create, update, retrieve, and secure data.

Examples

- MySQL
- Oracle
- SQL Server
- PostgreSQL

Functions

- Stores data
- Retrieves data
- Updates data
- Maintains security
- Reduces redundancy

2. Data Model

Definition

A **Data Model** is a collection of concepts used to describe the structure of a database, including data, relationships, and constraints.

Examples

- Relational Model
- Hierarchical Model
- Network Model
- ER Model

Purpose

- Organizes data.
- Defines relationships.
- Helps design databases.

3. ER Diagram

Definition

An **Entity-Relationship (ER) Diagram** is a graphical representation of entities, attributes, and relationships in a database.

Example

Student —— Enrolls —— Course

It is used during database design.

4. Entity

Definition

An **Entity** is any real-world object or thing about which data is stored.

Examples

- Student
- Employee

- Customer
- Product

5. Attribute

Definition

An **Attribute** describes the characteristics or properties of an entity.

Example
Student

- Student_ID
- Name
- Age
- Department

Here, Student is an entity, while Student_ID, Name, Age, and Department are attributes.

6. Relationship

Definition

A **Relationship** describes the association between two or more entities.

Example
Student Enrolls In Course
Customer Places Order

7. Schema

Definition
A **Schema** is the blueprint or structure of a database. It defines tables, attributes, relationships, and constraints.

Example
Student (Student_ID, Name, Department, Age)
This defines the database structure.

8. Instance

Definition
An **Instance** is the actual data stored in the database at a particular time.

Example

Student_ID	Name	Department	Age
101	jaya	CSE	20
102	Priya	ECE	21

When records are added, deleted, or modified, the instance changes, but the schema remains the same.

Relationship Among the Terms

- DBMS manages the database.
- A Data Model provides the rules for designing the database.
- The ER Diagram represents the database graphically.
- The ER Diagram contains Entities, Attributes, and Relationships.
- The database structure created from the design is called the Schema.
- The actual data stored according to the schema is called the Instance.

Conclusion

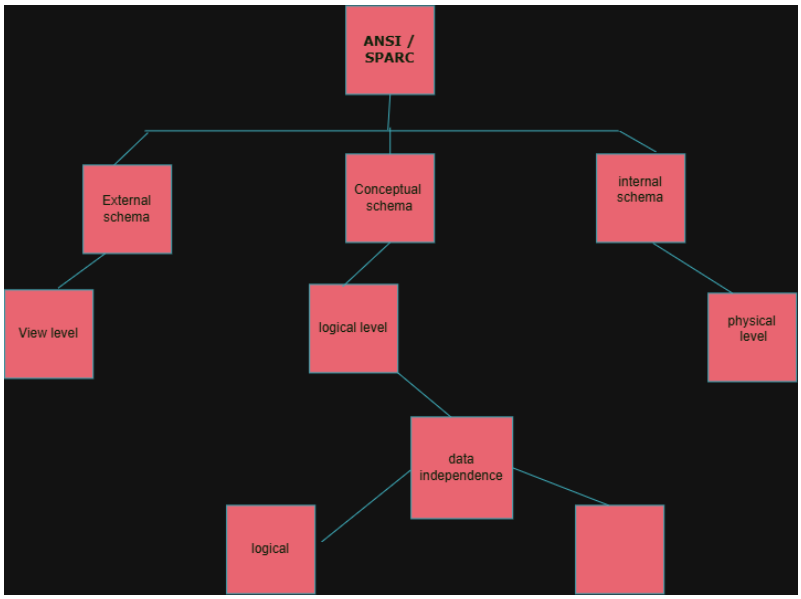
A DBMS uses a Data Model to organize data efficiently. During database design, an ER Diagram is created using Entities, Attributes, and Relationships. This design becomes the Schema, which defines the structure of the database. The Instance represents the current data stored in the database. Together, these concepts form the foundation of an effective Database Management System.

2. Create a concept map for the Three-Level ANSI/SPARC Architecture showing data independence at each level.

Introduction

The ANSI/SPARC Three-Level Architecture is a database architecture that separates the database into three levels: External Level, Conceptual Level, and Internal Level. This architecture provides data independence, allowing changes at one level without affecting the other levels.

Concept Map



Explanation of the Three Levels

1. External Level (View Level)

Definition

The **External Level** is the highest level of the database. It shows only the data required by a particular user and hides the remaining information.

Features

- Provides different views for different users.
- Improves security.
- Hides unnecessary data.

Example

- Student can view only **Name, Roll No., Department**.
- Faculty can view **Name, Marks, Attendance**.

2. Conceptual Level (Logical Level)

Definition

The **Conceptual Level** describes the complete logical structure of the database. It includes tables, attributes, relationships, and constraints.

Features

- Describes the entire database.
- Independent of physical storage.
- Used by database designers.

Example

Student(Student_ID, Name, Department)
Course(Course_ID, Course_Name)
Enrollment(Student_ID, Course_ID)

3. Internal Level (Physical Level)

Definition

The **Internal Level** describes how data is physically stored in the computer.

Features

- Defines storage methods.
- Includes files, indexes, and disk blocks.
- Managed b and y the DBMS.

Example

- Student records stored in **student.dat** file.
- B+ Tree index on **Student_ID**.

Data Independence

1. Logical Data Independence

Definition

Logical Data Independence is the ability to change the **Conceptual Schema** without affecting the **External Schema (User Views)**.

Example

Adding a new column (**Email**) to the Student table does not affect the existing user views.

2. Physical Data Independence

Definition

Physical Data Independence is the ability to change the **Internal Schema** without affecting the **Conceptual Schema**.

Example

Changing the storage method from **Sequential File** to **B+ Tree Index** does not affect the logical structure of the database.

Difference Between Logical and Physical Data Independence

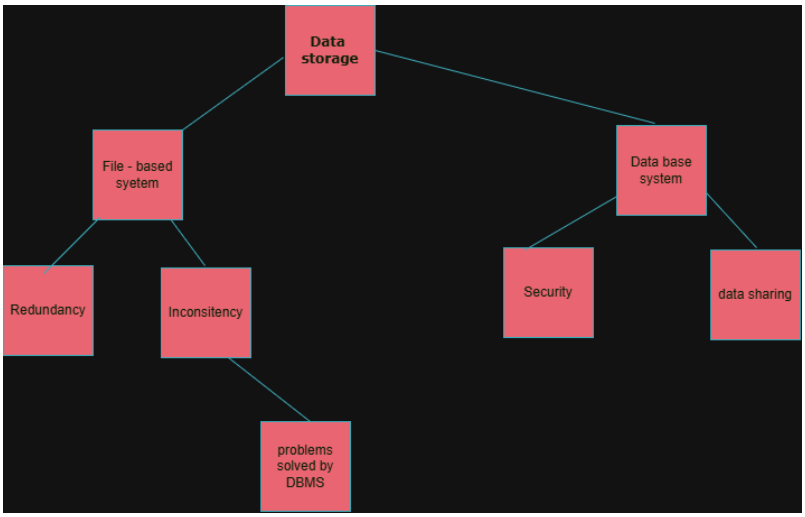
Logical Data Independence	Physical Data Independence
Changes in conceptual schema do not affect external schema.	Changes in internal schema do not affect conceptual schema.
More difficult to achieve.	Easier to achieve.
Example: Adding a new attribute to a table.	Example: Changing indexing or storage method.

Conclusion

The ANSI/SPARC Three-Level Architecture provides data abstraction through the External, Conceptual, and Internal Levels. It supports Logical Data Independence and Physical Data Independence, making databases more secure, flexible, and easier to maintain.

3. Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.

Concept Map



Introduction

A File-Based System stores data in separate files, whereas a Database Management System (DBMS) stores data in a centralized database. DBMS overcomes many limitations of file-based systems, such as data redundancy, inconsistency, and poor security.

File-Based System

Definition

A **File-Based System** stores data in separate files. Each application manages its own files independently.

Features

- Data stored in separate files.
- High data redundancy.
- Difficult to share data.
- Poor security.
- No centralized control.

Example
A college stores student details in one file and fee details in another file. Updating a student's information requires changes in multiple files.

Database Management System (DBMS)

Definition

A **Database Management System (DBMS)** is software that stores and manages data in a centralized database. It provides security, consistency, and efficient data access.

Features

- Centralized data storage.
- Reduces redundancy.
- Provides data security.
- Supports backup and recovery.
- Allows multiple users to access data.

Example
In a college DBMS, student, faculty, and fee information are stored in one database. Any update is reflected throughout the system.

Problems Solved by DBMS

Problem in File-Based System	Solution Provided by DBMS
Data redundancy	Reduces duplicate data
Data inconsistency	Maintains consistent data
Poor security	Provides user authentication and access control
Difficult data sharing	Enables easy sharing among users
No backup and recovery	Supports backup and recovery
Data isolation	Stores all data in a centralized database
Difficult maintenance	Simplifies data management

Difference Between File-Based System and DBMS

File-Based System	Database Management System (DBMS)
Stores data in separate files.	Stores data in a centralized database.
High data redundancy.	Reduced data redundancy.
Low security.	High security.
Difficult data sharing.	Easy data sharing.
No backup and recovery.	Backup and recovery available.
Data inconsistency may occur.	Ensures data consistency.

Conclusion

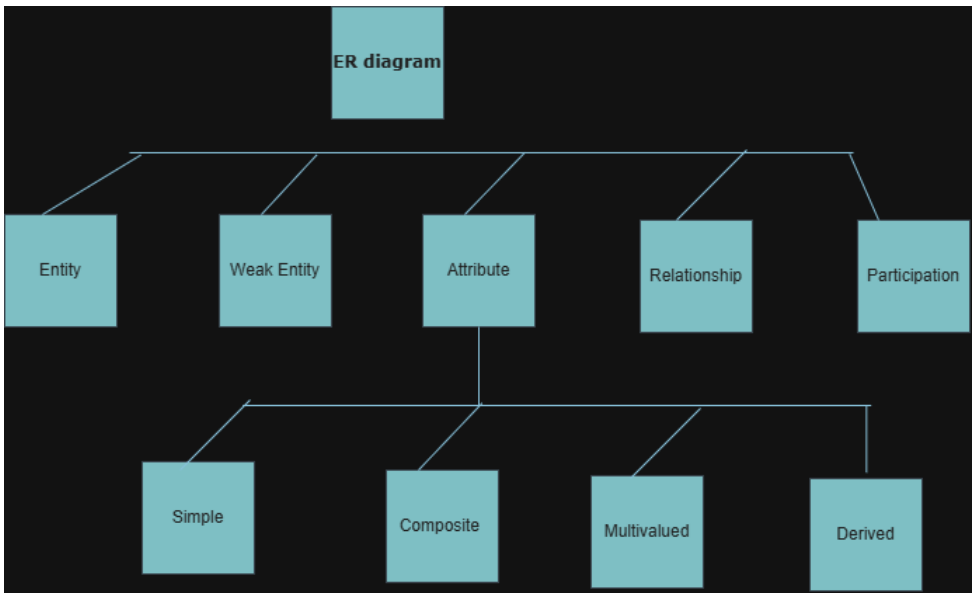
A **File-Based System** is suitable for small applications but has several limitations, including data redundancy and inconsistency. A **DBMS** solves these problems by providing centralized storage, better security, data consistency, backup and recovery, and efficient data management, making it suitable for modern applications.

4 . Design a concept map for an ER Diagram showing Entity, Weak Entity, Attribute Types, Relationship Types, and Participation Constraints.

Introduction

An Entity-Relationship (ER) Diagram is a graphical representation of a database. It shows entities, attributes, relationships, weak entities, and participation constraints, helping database designers organize and understand data efficiently.

Concept Map



1. Entity

Definition

An **Entity** is a real-world object or person about which data is stored.

Example

- Student
- Employee
- Customer

2. Weak Entity

Definition
A **Weak Entity** is an entity that cannot exist without a strong entity. It does not have its own primary key.

Example
- Dependent (depends on Employee)
- Order Item (depends on Order)

3. Attribute Types

Definition
Attributes describe the properties of an entity.

Types of Attributes
- **Simple Attribute:-** Cannot be divided (Age)
- **Composite Attribute:-** Can be divided (Name → First Name, Last Name)
- **Single-valued Attribute:-** One value (Roll Number)
- **Multi-valued Attribute:** Multiple values (Phone Numbers)
- **Derived Attribute:** Calculated from another attribute (Age from Date of Birth)

4. Relationship Types

Definition
A **Relationship** shows how two or more entities are connected.

Types
- **One-to-One (1:1)** – One employee manages one department.
- **One-to-Many (1:M)** – One customer has many accounts.
- **Many-to-Many (M:N)** – Many students enroll in many courses.

5. Participation Constraints

Definition
Participation constraints specify whether all entities must participate in a relationship.

Types
- **Total Participation:** Every entity must participate.
- **Partial Participation:** Participation is optional.

Example

- **Total:** Every employee must belong to a department.

- **Partial:** A customer may or may not apply for a loan.

Conclusion

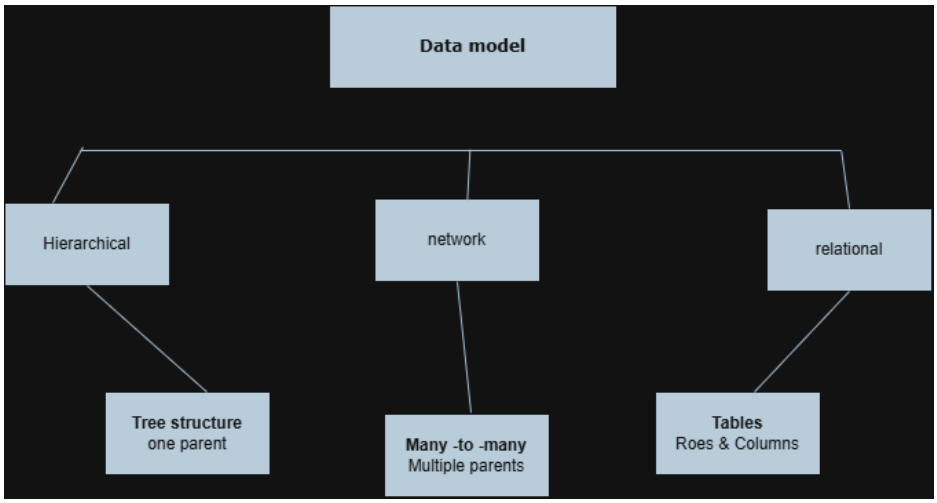
The ER Diagram represents the structure of a database using entities, weak entities, attributes, relationships, and participation constraints. These concepts help in designing efficient, organized, and reliable databases.

5. Construct a concept map comparing Hierarchical, Network, and Relational Data Models with their key properties.

Introduction

A Data Model defines how data is organized, stored, and accessed in a database. The three common database models are the Hierarchical Model, Network Model, and Relational Model. Each model has a different way of organizing data.

Concept Map



1. Hierarchical Model

Definition

The **Hierarchical Model** organizes data in a **tree structure**. Each child has only one parent, creating a **one-to-many** relationship.

Key Properties

- Tree structure.
- Parent-child relationship.
- Supports one-to-many relationships.
- Fast data retrieval.

Example

Organization chart, Folder structure.

2. Network Model

Definition

The **Network Model** organizes data in a **graph structure**, allowing a child to have multiple parents. It supports **many-to-many** relationships.

Key Properties

- Graph structure.
- Multiple parent-child relationships.
- Supports many-to-many relationships.
- More flexible than the hierarchical model.

Example

Airline reservation system, Social networks.

3. Relational Model

Definition

The **Relational Model** stores data in **tables** consisting of rows and columns. Relationships are created using **Primary Keys** and **Foreign Keys**.

Key Properties

- Table-based structure.
- Uses rows and columns.
- Supports SQL.
- Easy to design and maintain.
- Reduces data redundancy.

Example

Student Management System, Banking System.

Comparison of Data Models

Hierarchical Model	Network Model	Relational Model
Tree structure	Graph structure	Table structure
One-to-Many	Many-to-Many	Uses Primary & Foreign Keys
One parent for each child	Multiple parents for a child	Data stored in rows and columns
Less flexible	More flexible	Easy to use and manage

Conclusion

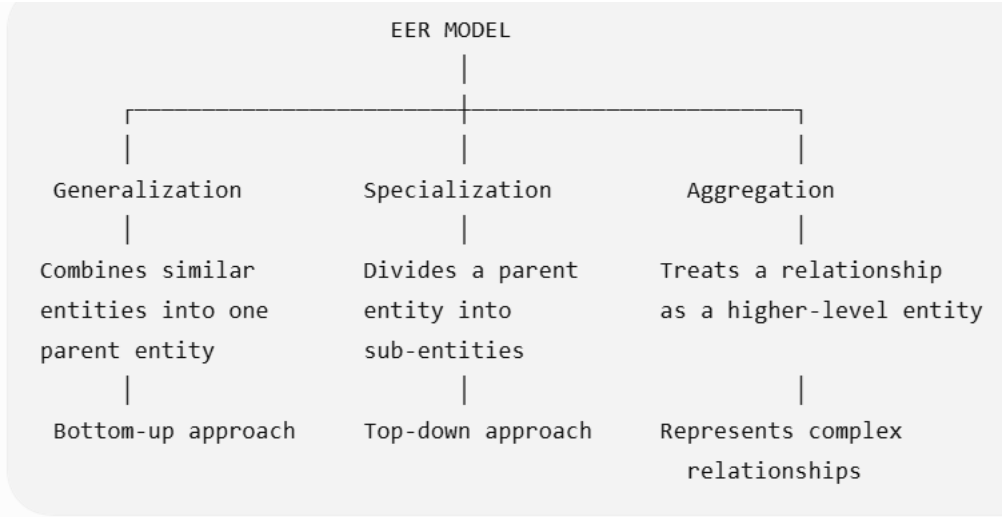
The Hierarchical Model is suitable for tree-like data, the Network Model is suitable for complex many-to-many relationships, and the Relational Model is the most widely used because it organizes data in tables, supports SQL, and provides efficient data management.

6. Develop a concept map for the Extended Entity-Relationship (EER) Model showing Generalization, Specialization, and Aggregation.

Introduction

The Extended Entity-Relationship (EER) Model is an advanced version of the ER Model. It includes additional concepts such as Generalization, Specialization, and Aggregation to represent complex real-world database applications more effectively.

Concept Map



1. Generalization

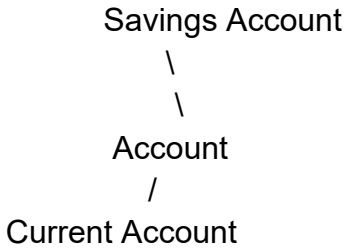
Definition

Generalization is the process of combining two or more similar lower-level entities into a single higher-level entity based on their common features.

Features

- Bottom-up approach.
- Combines similar entities.
- Creates a common parent entity.
- Reduces data redundancy.

Example



2. Specialization

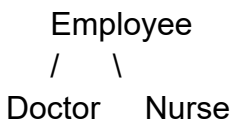
Definition

Specialization is the process of dividing one higher-level entity into multiple lower-level entities based on specific characteristics.

Features

- Top-down approach.
- Creates specialized child entities.
- Child entities inherit attributes from the parent entity.

Example



3. Aggregation

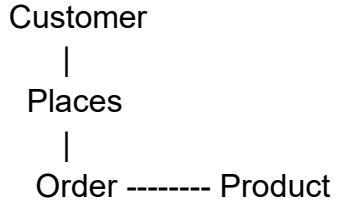
Definition

Aggregation is the process of treating a relationship between entities as a higher-level entity so that it can participate in another relationship.

Features

- Represents complex relationships.
- Treats a relationship as an entity.
- Simplifies database design.

Example



Here, the **Order** relationship is treated as an entity connected to **Product**.

Comparison

Generalization	Specialization	Aggregation
Combines lower-level entities.	Divides a higher-level entity.	Treats a relationship as an entity.
Bottom-up approach.	Top-down approach.	Used for complex relationships.
Example: Savings & Current → Account	Example: Employee → Doctor, Nurse	Example: Customer places Order

Conclusion

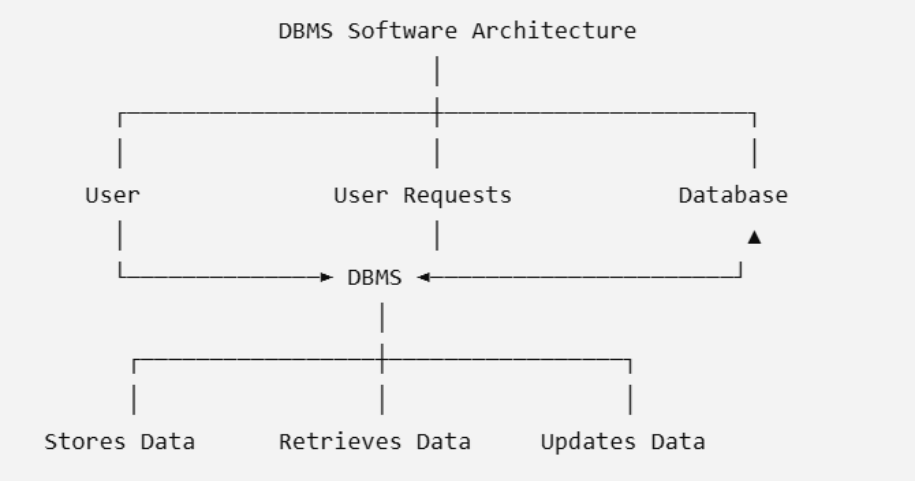
The EER Model extends the ER model by introducing Generalization, Specialization, and Aggregation. These concepts improve database design by reducing redundancy, representing inheritance, and handling complex relationships efficiently.

7. Create a visual concept map for DBMS Software Architecture, illustrating the interaction between the User, User Requests, DBMS, and Database.

Introduction

DBMS Software Architecture shows how users interact with the database through the Database Management System (DBMS). The user sends requests to the DBMS, which processes them and retrieves or updates data in the database.

Concept Map



1. User

Definition
A **User** is a person or application that accesses the database.

Examples

- Student
- Employee
- Administrator

2. User Requests

Definition
A **User Request** is an instruction sent by the user to perform operations such as inserting, updating, deleting, or retrieving data.

Examples
- Insert a new student record.
- Search employee details.
- Update customer information.

3. DBMS

Definition
A **Database Management System (DBMS)** is software that receives user requests, processes them, and communicates with the database.

Functions
- Stores data.
- Retrieves data.
- Updates records.
- Deletes records.
- Provides security.

Examples
- MySQL
- Oracle
- SQL Server
- PostgreSQL

4. Database

Definition
A Database is an organized collection of related data stored electronically.

Examples
- Student Database
- Hospital Database
- Banking Database

Working of DBMS Software Architecture

1. The User sends a request.
2. The DBMS receives and processes the request.
3. The DBMS accesses the Database.
4. The Database returns the required data.
5. The DBMS sends the result back to the User.

Conclusion

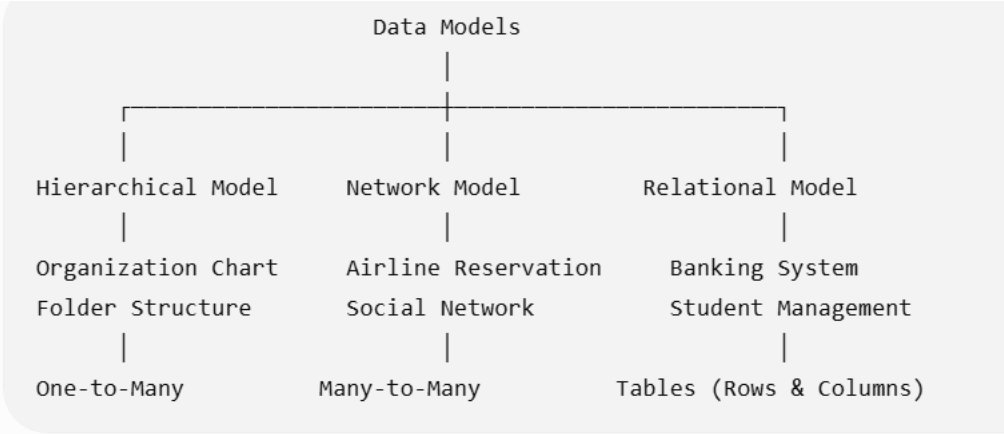
The DBMS Software Architecture acts as a bridge between the User and the Database. It processes user requests, manages data efficiently, ensures security, and provides accurate information whenever required.

8. Map the data models discussed in class to real-world applications and explain their suitability.

Introduction

A Data Model defines how data is organized, stored, and managed in a database. Different data models are suitable for different real-world applications based on the type of data and relationships.

Concept Map



1. Hierarchical Model

Definition

The Hierarchical Model organizes data in a tree structure where each child has only one parent.

Real-World Applications

- Organization Chart
- Folder Structure
- Family Tree

Suitability

- Best for one-to-many relationships.
- Fast and simple for hierarchical data.

2. Network Model

Definition

The Network Model organizes data as a graph, allowing multiple parent-child relationships.

Real-World Applications

- Airline Reservation System
- Railway Reservation System
- Social Networking Sites

Suitability

- Supports many-to-many relationships.
- Suitable for complex data structures.

3. Relational Model

Definition

The Relational Model stores data in tables using rows and columns. Relationships are created using Primary Keys and Foreign Keys.

Real-World Applications

- Banking System
- Hospital Management System
- Student Management System
- Library Management System

Suitability

- Easy to design and maintain.
- Reduces data redundancy.
- Supports SQL for efficient data retrieval.

Comparison of Data Models

Data Model	Real-World Application	Suitability
Hierarchical Model	Organization Chart, Folder Structure	Best for one-to-many relationships
Network Model	Airline Reservation, Social Networks	Best for many-to-many relationships
Relational Model	Banking, Hospital, Student Management	Best for table-based data and SQL

Conclusion

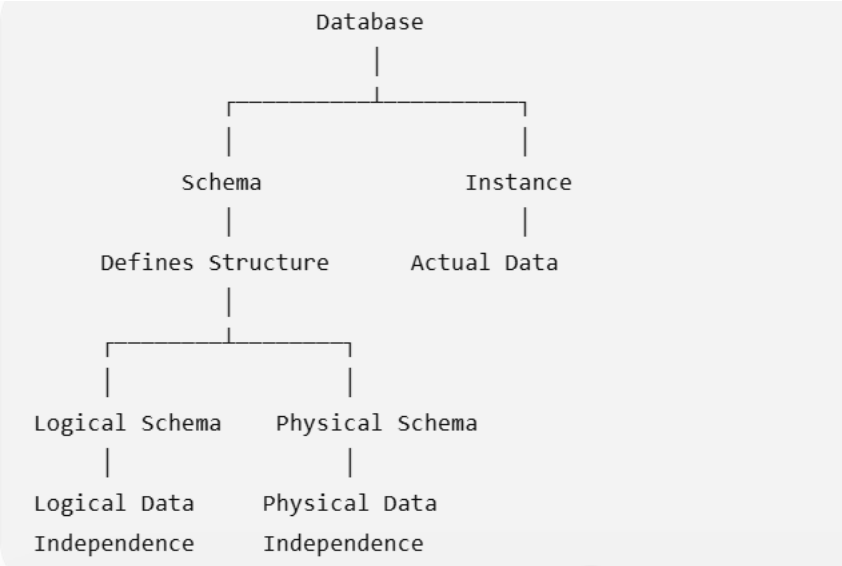
Different data models are used for different applications. The Hierarchical Model is suitable for tree-structured data, the Network Model is suitable for complex relationships, and the Relational Model is the most widely used because it organizes data efficiently using tables and supports SQL.

9. Construct a concept map for Schema and Instance, showing their relation to Physical and Logical Data Independence.

Introduction

Schema defines the structure of a database, while an Instance represents the actual data stored in the database at a particular time. The Three-Schema Architecture supports Logical Data Independence and Physical Data Independence, allowing changes at one level without affecting the other levels.

Concept Map



1. Schema

Definition

A Schema is the blueprint or structure of a database. It defines tables, attributes, relationships, and constraints.

Example

Student (Student_ID, Name, Department)

2. Instance

Definition

An **Instance** is the actual data stored in the database at a specific time.

Example

Student_ID	Name	Department
101	Jaya	CSE
102	Priya	ECE

3. Logical Data Independence

Definition

Logical Data Independence is the ability to change the logical schema without affecting the user views (external schema).

Example

Adding a new column (**Email**) to the Student table without changing existing user applications.

4. Physical Data Independence

Definition

Physical Data Independence is the ability to change the physical storag**e** without affecting the logical schema.

Example

Changing the storage method from sequential files to indexed files without changing the table structure.

Relationship

- **Schema** defines the database structure.
- **Instance** stores the current data.
- **Logical Data Independence** protects user views when the logical schema changes.
- **Physical Data Independence** protects the logical schema when storage methods change.

Conclusion

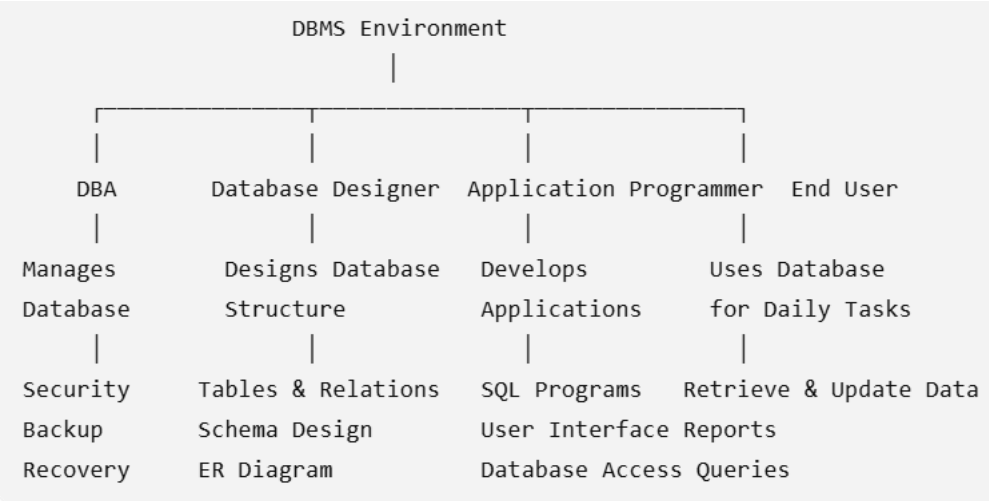
The Schema provides the structure of the database, while the Instance contains the actual data. Logical Data Independence and Physical Data Independence make the database flexible by allowing changes at different levels without affecting other parts of the system.

10. Design a concept map for the roles and responsibilities in a DBMS environment, including DBA, End User, Application Programmer, and Database Designer.

Introduction

A DBMS environment involves different people who work together to design, manage, maintain, and use the database. The main roles are Database Administrator (DBA), Database Designer, Application Programmer, and End User.

Concept Map



1. Database Administrator (DBA)

Definition

A Database Administrator (DBA**)** is responsible for managing and maintaining the database.

Responsibilities

- Installs and configures the DBMS.
- Manages database security.
- Performs backup and recovery.
- Monitors database performance.
- Controls user access.

2. Database Designer

Definition

A Database Designer designs the database structure based on user requirements.

Responsibilities

- Creates the database schema.
- Designs ER diagrams.
- Defines tables and relationships.
- Selects keys and constraints.

3. Application Programmer

Definition

An Application Programmer develops software applications that interact with the database.

Responsibilities

- Writes SQL queries.
- Develops forms and reports.
- Creates user interfaces.
- Connects applications to the database.

4. End User

Definition

An End User is the person who uses the database through an application.

Responsibilities

- Retrieves data.
- Inserts and updates records.
- Generates reports.
- Uses database applications for daily tasks.

Comparison of Roles

Role	Main Responsibility
DBA	Manages, secures, and maintains the database.
Database Designer	Designs the database structure and schema.
Application Programmer	Develops database applications and SQL programs.
End User	Uses the database to perform daily operations.

Conclusion

A DBMS environment consists of different roles working together. The Database Designer creates the database, the DBA manages and secures it, the Application Programmer develops database applications, and the End User accesses and uses the database efficiently.